计算机工程实践 PPT4 补充

蓝敏源

brucelan1777@sjtu.edu.cn

2026 年 03 月 23 日

摘要：本文档主要补充了 OpenCV 的基础函数的介绍，同时简单介绍了 RGB 空间和 HSV 空间以及霍夫圆检测。

关键字： OpenCV；RGB；HSV

一 OpenCV

OpenCV 是一个开源的计算机视觉与图像处理库

由 Intel，支持 C++、Python、Java 等多种语言

可以跨平台运行，支持 Windows、MacOS、Linux、Android、IOS 等平台。

1.1 图像处理

1.1.1 基础功能

读取图片：

```
cv2.imread(path)
```

彩色转灰度

```
cv2.cvtColor（img,cv2.COLOR_BRG2GRAY)
```

色彩空间转换

```
cv.cvtColor(frame, cv.COLOR_RGB2BGR)
```

保存图片

```
cv2.imwrite(save_path, name)
```

图像裁切

```
cropped_img = img[row_start:row_end, col_start:col_end]
```

1.1.2 图像缩放

缩放图像时，本质上是将原图像的像素映射到目标图像的像素网格上，但由于两者的网格并不一定对齐，有时需要通过插值来计算目标像素的值。不同的插值方法决定了最终图像的质量、平滑度和计算速度。

1.1.2.1 基本用法

```
import cv2

img = cv2.imread('input.jpg')
# 指定目标尺寸（宽度，高度）
resized = cv2.resize(img, (800, 600))
# 或者指定缩放因子
resized = cv2.resize(img, None, fx=0.5, fy=0.5)
```

1.1.2.2 插值方法[1]

OpenCV 提供了多种插值标志，通过 interpolation 参数指定。下面按“放大”和“缩小”两种场景分别说明其特点。

- 最近邻插值
  - 标志：cv2.INTER_NEAREST
  - 原理：取目标像素位置最近的原像素值，不考虑周围像素。
  - 适用场景：图像缩放后仍希望保持像素风格（如像素艺术）、二值图像、或需要极快速度时。
  - 速度：最快
  - 质量：最差（出现锯齿、马赛克）
- 双线性插值
  - 标志：cv2.INTER_LINEAR
  - 原理：取目标像素周围 2×2 邻域，进行两次线性加权平均。
  - 适用场景：默认方法，适合大多数场景，尤其是放大图像时能获得平滑结果。
  - 速度：快
  - 质量：较好
- 区域插值
  - 标志：cv2.INTER_AREA
  - 原理：基于像素面积关系进行重采样。对缩小图像效果最好，可避免波纹（摩尔纹）。
  - 适用场景：缩小图像的首选。当 fx<1 或 fy<1 时推荐使用。
  - 速度：中等
  - 质量：很好（缩小后细节清晰）
- 双三次插值
  - 标志：cv2.INTER_CUBIC
  - 原理：取周围 4×4 邻域，用三次样条函数加权平均。
  - 适用场景：放大图像时追求更高质量，比双线性更平滑但稍慢。
  - 速度：较慢
  - 质量：很好
- Lanczos 插值

- 标志：cv2.INTER_LANCZOS4
- 原理：使用 Lanczos 核（8×8 邻域）进行更精确的 sinc 函数近似。
- 适用场景：放大图像时最高质量，适合对细节要求极高的场合。
- 速度：最慢
- 质量：最好（但可能由轻微振铃）

1.1.3 图像平滑与去噪

图像平滑与去噪是图像预处理中最基础也是最重要的步骤之一。其核心目标是在保留图像重要结构（如边缘、纹理）的前提下，抑制不必要的噪声。OpenCV 提供了多种平滑（模糊）滤波器，它们通过不同的邻域加权策略实现这一目标。

- 高斯模糊（Gaussian Blur）
  - 原理：高斯模糊使用符合二维高斯分布的权重核进行加权平均。距离中心越近的像素权重越大，距离越远的权重越小，因此能更自然地保留边缘（相对均值而言）。核由高斯函数 $G(X,Y)=\left(\frac{1}{2\pi\sigma^2}\right)e^{\frac{-x^2+y^2}{2}\sigma^2}$ 生成;关键参数为标准差 σ 和核大小 ksize。σ 越大，模糊程度越强；核大小需足够大以覆盖主要权重范围。
  - ```
    dst = cv2.GaussianBlur(src, ksize, sigmaX, sigmaY=0, borderType=None)
    ```
    - *ksize*：核大小（必须为奇数，可为 *(0,0)* 表示由 σ 自动计算）。
    - *sigmaX*：X 方向的标准差。若为 *0*，则根据核大小自动计算。
    - *sigmaY*：Y 方向标准差，默认与 *sigmaX* 相同。
  - 优点：平滑效果自然，比均值模糊更好地保留边缘。
  - 缺点：仍是线性滤波，对椒盐噪声等非高斯噪声效果不佳。
  - 适用场景：通用降噪、边缘检测前的预处理、图像金字塔构建、美颜磨皮（配合掩膜）。
- 均值模糊（Mean Blur / Average Blur）
  - 原理：均值模糊是最简单的线性滤波器。它用一个所有系数相等的卷积核对图像进行卷积，即计算核窗口内所有像素的平均值来替换中心像素。核大小：ksize = (width, height)，通常为奇数，如 (3,3)、(5,5)。核的每个元素为 1/(ksize.width * ksize.height)。
  - ```
    dst = cv2.blur(src, ksize, anchor=None, borderType=None)
    ```
    - src：输入图像（可为灰度或彩色）。
    - ksize：卷积核大小，如 (5,5)。
    - anchor：锚点（默认 (-1,-1) 表示核中心）。
  - 优点：计算简单、速度快。
  - 缺点：会同时平滑噪声和边缘，导致图像模糊、细节丢失。
  - 适用场景：对边缘要求不高的快速降噪、图像预处理（如缩小前的抗混叠）。
- 中值模糊（Median Blur）

- 原理：中值滤波是一种非线性滤波器。它将核窗口内所有像素按灰度值排序，取中位数作为输出。这种操作对椒盐噪声（随机黑白点）有极佳的抑制效果，同时能较好地保留边缘。
- ```
  dst = cv2.medianBlur(src, ksize)
  ```
  - ksize：核的直径（必须是大于 1 的奇数，如 3、5、7）。注意：这里是边长，不是元组。
- 优点：有效去除椒盐噪声、斑点噪声，边缘保留能力优于线性滤波。
- 缺点：计算量较大（需排序），核越大越耗时。
- 适用场景：存在椒盐噪声的图像（如扫描文档、老照片）、医学图像预处理、图像修复中的初始去噪。

1.1.4 图像边缘检测

边缘检测是图像处理和计算机视觉中的核心技术之一，其目标是识别图像中亮度变化剧烈的像素点，这些点通常对应物体的边界、轮廓或纹理突变。边缘检测可以从图像中提取出最重要的结构信息，为后续的物体识别、图像分割、特征匹配等任务奠定基础。

OpenCV 提供了多种边缘检测算子，从经典的 Sobel、Laplacian 到功能强大的 Canny 边缘检测器。

1.1.4.1 边缘检测的基本原理

边缘在数学上表现为图像灰度函数的局部梯度较大的区域。对于一维信号，边缘对应导数的峰值；对于二维图像，我们可以通过计算梯度幅值和梯度方向来定位边缘。

梯度：$\nabla F = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right)$

梯度方向指向灰度变化最快的方向，梯度幅值表示变化的剧烈程度。

梯度幅值：$|\nabla F| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$

通常作为边缘强度的度量。

梯度方向$\theta = \arctan\left(\frac{\frac{\partial f}{\partial x}}{\frac{\partial f}{\partial y}}\right)$

用于后续的非极大值抑制（如 Canny 算法）。

由于数字图像是离散的，梯度通常通过卷积（即与差分算子卷积）来近似计算。

1.1.4.2 常用边缘检测算子

- Sobel 算子

Sobel 算子是一种一阶微分算子，它使用两个 3×3 卷积核分别计算图像在水平和垂直方向上的近似梯度：

水平方向（检测垂直边缘）：

垂直方向（检测水平边缘）：

然后计算梯度幅值：$G = \sqrt{G_x^2 + G_y^2}$（常用近似：$|G_x| + |G_y|$ 以提高速度）。

```
#ddepth：输出图像深度，通常设为 cv2.CV_64F 以避免截断。
#ksize：核大小，可选 1、3、5、7。核越大，平滑效果越强，但边缘定位精度下降。
```
```

```
sobel_x = cv2.Sobel(src, ddepth, dx=1, dy=0, ksize=3)
sobel_y = cv2.Sobel(src, ddepth, dx=0, dy=1, ksize=3)
grad = cv2.magnitude(sobel_x, sobel_y) #计算幅值
```

特点:对噪声有一定平滑作用(因为核中有权重);能同时计算梯度方向和幅值;边缘较粗,定位精度中等。

- Laplacian 算子

Laplacian 算子是一种二阶微分算子,它计算图像的二阶导数(拉普拉斯算子),对灰度突变的中心(即边缘的过零点)响应最大。

拉普拉斯算子定义为:$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$。

常用离散核(3×3):$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$ 或 $\begin{pmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{pmatrix}$

```
#ksize 可以是 1、3、5、7(奇数)。若为 1,则使用前述的 3×3 核;若更大,则使用更大尺寸的拉普拉斯核。
laplacian = cv2.Laplacian(src, ddepth, ksize=3)
```

特点:对噪声非常敏感,通常需先对图像进行平滑(如高斯模糊);**输出的是边缘的强度,但通常需要过零点检测才能得到单像素边缘**;常用于图像锐化(原图减去拉普拉斯结果)。

- Canny 边缘检测

Canny 边缘检测器被公认为最优的边缘检测算法之一,它通过一系列步骤实现了低错误率、高定位精度、单像素响应的目标。

算法步骤:

1. 高斯滤波:使用高斯模糊平滑图像,去除噪声。
2. **计算梯度幅值与方向:通常用** Sobel 或 Scharr 算子计算 G_x、G_y,得到梯度幅值 G 和方向 θ。
3. 非极大值抑制(NMS):在梯度方向上,将每个像素的梯度幅值与相邻像素(沿梯度方向)进行比较,只保留局部最大值,从而"瘦身"边缘,得到单像素宽度的候选边缘。
4. 双阈值检测:设定两个阈值 high_thresh 和 low_thresh。
   (1) 幅值大于 high_thresh 的像素为强边缘。
   (2) 幅值介于两者之间的为弱边缘。
   (3) 幅值小于 low_thresh 的被抑制。
5. 边缘连接:通过滞后阈值,只保留那些与强边缘连通的弱边缘,其他弱边缘被丢弃。这样能确保边缘的连续性。

```
#threshold1:低阈值
#threshold2:高阈值
#apertureSize:Sobel 算子的核大小(默认 3)
#L2gradient:是否使用更精确的 L2 范数($sqrt(G_x^2+G_y^2)$)计算幅值。设为 True 可提高
```

```
精度，但速度稍慢
edges = cv2.Canny(image, threshold1, threshold2, apertureSize=3, L2gradient=False)
```

参数调优经验 阈值：通常高阈值取低阈值的 2 3 倍。例如 (50, 150) 或 (100, 200)。低阈值越低，检测到的边缘越多；高阈值越高，边缘越少且越显著。

图像预处理：若噪声较多，可先使用高斯模糊（cv2.GaussianBlur）降噪，再应用 Canny。但注意 Canny 内部已包含高斯滤波，因此外部滤波可选。

核大小：增大 apertureSize 会使 Sobel 核更大，对噪声更鲁棒，但边缘定位精度下降。

特点：鲁棒性强，能检测出连续、细化的边缘;参数较多，需根据图像内容调整,是目前最常用的边缘检测方法。

1.2 颜色识别

RGB 和 HSV 是两种最常用的色彩空间（颜色模型），它们本质上是同一物理颜色（可见光谱）的不同数学表达方式。RGB 面向硬件（屏幕、相机），而 HSV 面向人类直觉（选色、调色）。

1.2.1 RGB 空间

RGB 是 加色模型，基于人眼三种视锥细胞（红、绿、蓝）的生理原理。

原理：在黑色背景上，通过混合不同强度的 R（红）、G（绿）、B（蓝）三原色光来产生颜色。当三者强度都为最大值（如 255,255,255）时，叠加为白色；都为最小值（0,0,0）时，为黑色。

结构：它是一个立方体。三个坐标轴分别代表 R、G、B 的值，原点为黑，对角顶点为白。

优点：

- 硬件友好：这是所有显示器、扫描仪、摄像头直接使用的原生格式。
- 线性相关：在物理光照计算中，RGB 值的变化通常与光强呈线性关系，适合图形渲染管线。

缺点：

- 不直观：对人类来说，很难通过“增加 B 分量，减少 G 分量”的数值调整，来精确找到想要的“土黄色”或“深红色”。
- 通道相关性高：调整亮度或饱和度时，必须同时修改 R、G、B 三个数值，且比例复杂。例如，将红色调暗一点，不仅是减少 R，往往还需要微调 G 和 B 来维持色相不变。
- 受环境影响强，环境亮度的改变会导致 RGB 三个通道的值发生改变

RGB 颜色空间适合于显示系统，却并不适合于图像处理。

1.2.2 HSV 空间

HSV 是一种 主观感知模型，它将颜色分解为人类认知颜色的三个基本属性。它通常被描绘成一个倒圆锥体或六棱锥。在 HSV 颜色空间下，比 BGR 更容易跟踪某种颜色的物体，常用于分割指定颜色的物体。

HSV 表达彩色图像的方式由三个部分组成：

- Hue（色调、色相）决定是什么颜色（红、橙、绿等）。0°为红色，120°为绿色，240°为蓝色。它沿着圆锥体的外圆周变化。

- Saturation（饱和度、色彩纯净度）决定颜色的鲜艳程度。0% 是圆锥中心轴，颜色为灰、白、黑（无彩色）；100% 是圆锥外表面，颜色最纯、最饱和。对应半径
- Value（明度）决定颜色的亮暗。0% 是黑色；100% 最亮。对应高度

优点：

- 直觉性强：要改变颜色，只需调整 H；要褪色，只需降低 S；要变暗，只需降低 V。
- 易于分离光照信息：在图像处理中（如肤色检测、物体追踪），通常只提取 H 通道，因为 V（亮度）的变化不会影响 H（色相）的识别，这使得算法对光照阴影具有较强的鲁棒性。

```
image = cv2.imread(img_path)
cv2.imshow('img', image)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
cv2.imshow('hsv', hsv[:,:,0])
(thresh, im_bw) = cv2.threshold(hsv[:,:,0], 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)
cv2.imshow('otsu', im_bw)
```

1.3 霍夫圆检测

梯度霍夫圆检测的原理 梯度霍夫圆检测主要分为两个阶段：

1. 圆心检测
   (1) 边缘提取：通常使用 Canny 边缘检测获得二值边缘图像。
   (2) 计算梯度方向：对每个边缘点，计算其梯度方向（即该点切线的垂直方向）。
   (3) 投票累加：对于每个边缘点，沿其梯度方向累加“可能圆心”的投票。由于圆心位于边缘点的法线方向上，所有属于同一个圆的边缘点的法线会相交于圆心。累加器（二维数组）中峰值最高的位置就是候选圆心。

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n-1)
print(factorial(5))  # 输出: 120
```

参考文献

[1] . OpenCV_InterpolationFlags[EB/OL]. https://docs.opencv.org/4.0.0/da/d54/group__imgproc__transform.html#ga5bb5a1fea74ea38e1a5445ca803ff121.